

ARUS Backend — Code Quality Fixes

Generated: 2026-03-26

Summary

This package contains fixes for **all 13 issues** identified in the code quality review. Each fix is either a full replacement file or a targeted patch with instructions.

Quick Start

1. **Install new dependency** (for C1 fix):

```
npm install bcrypt
npm install -D @types/bcrypt
```

2. **Copy replacement files** over the originals in your Replit project:

fixes/server/domains/system-admin/routes/auth-routes.ts	→ server/domains/s
fixes/server/routes/analytics/helpers.ts	→ server/routes/a
fixes/server/domains/alerts/routes.ts	→ server/domains/
fixes/server/routes/insights-routes.ts	→ server/routes/i
fixes/server/routes/telemetry-ingestion-routes.ts	→ server/routes/t
fixes/server/routes/telemetry-dlq-routes.ts	→ server/routes/t
fixes/server/domains/dtc/routes.ts	→ server/domains/
fixes/server/domains/permissions/routes.ts	→ server/domains/

3. **Apply manual patches** for M5 (kb-routes.ts) — see below.
 4. **Add batch storage method** for H2 — see below.
 5. **Re-hash your admin password** after deploying C1 fix — see below.
-

Fix Details by Finding ID

C1 — Admin Password Bcrypt Hashing (CRITICAL)

File: `server/domains/system-admin/routes/auth-routes.ts` **Action:** Full replacement file provided.

Changes:

- `bcrypt.hash(password, 12)` on setup and password change
- `bcrypt.compare(password, storedHash)` on verify
- Extracted `createAdminSession()` helper to reduce duplication

⚠ **Post-deploy step:** After deploying this fix, your existing plaintext `ADMIN_TOKEN` in `.env` will fail `bcrypt.compare`. You must either:

- Delete `ADMIN_TOKEN` from `.env` and re-run `/api/admin/auth/setup` with your password
- Or manually set `ADMIN_TOKEN` to a bcrypt hash:

```
node -e "require('bcrypt').hash('your-password', 12).then(h => console.log(h))"
```

Then paste the output as the `ADMIN_TOKEN` value.

C2 — Org-Scoping Gaps (CRITICAL)

Files:

- `server/routes/analytics/helpers.ts` — Full replacement
- `server/domains/alerts/routes.ts` — Full replacement
- `server/routes/insights-routes.ts` — Full replacement

Changes:

- `getOrgId()` in `analytics/helpers.ts` now returns `null` and sends 401 if `orgId` is missing (previously returned "default-org" silently)
- All analytics route callers must check: `const orgId = getOrgId(req, res); if (!orgId) return;`
- Alert `listNotifications()` and `listSuppressions()` now receive and pass `orgId`
- Insights queries now filter by `eq(actionableInsights.orgId, orgId)` everywhere

⚠ **Storage layer change needed for alerts:** Your `alertsService.listNotifications()`

and `alertsService.listSuppressions()` must accept an `orgId` parameter and use it in their WHERE clauses. Add the parameter to the service interface.

⚠ Analytics callers update needed: Every file that calls `getOrgId()` must now handle the null return. Check these files:

- `server/routes/analytics/health-metrics.ts`
- `server/routes/analytics/predictions.ts`
- `server/routes/analytics/costs-and-feedback.ts`
- `server/routes/analytics/model-governance.ts`
- `server/routes/analytics/cache-reconciliation.ts`

In each, change:

```
// BEFORE:
const orgId = getOrgId(req, res);

// AFTER:
const orgId = getOrgId(req, res);
if (!orgId) return; // 401 already sent
```

H1 — Telemetry Routes Missing withErrorHandling (HIGH)

Files:

- `server/routes/telemetry-ingestion-routes.ts` — Full replacement
- `server/routes/telemetry-dlq-routes.ts` — Full replacement

Changes:

- All routes migrated from raw `try/catch` to `withErrorHandling("name", handler)`
- Error responses now use `{ message: "..."} shape (consistent with rest of API)` instead of `{ error: "..."} shape`
- Logging goes through centralized error handler

H2 — N+1 Queries in DTC Routes (HIGH)

File: `server/domains/dtc/routes.ts` — Full replacement

Changes:

- `GET /api/dtc/active` now checks for `storage.getActiveDtcsBatch()` and uses it if available (single query with IN clause)
- Falls back to chunked parallel queries (50 at a time) if batch method doesn't exist
- You should add this method to your storage layer:

```
// In your storage implementation:
async getActiveDtcsBatch(equipmentIds: string[], orgId: string) {
  return db
    .select()
    .from(dtcFaults)
    .leftJoin(dtcDefinitions, /* ... */)
    .where(
      and(
        inArray(dtcFaults.equipmentId, equipmentIds),
        eq(dtcFaults.orgId, orgId),
        eq(dtcFaults.status, 'active')
      )
    );
}
```

Similarly for fatigue routes (H2 also applies to `server/domains/stcw-rest/routes/fatigue.ts`): Add `storage.getCrewRestRangeBatch(crewIds, startDate, endDate)` that fetches all crew rest data in a single query. The fix pattern is the same — check for batch method, use if available, chunk-parallel fallback otherwise.

H3 — Dynamic Imports Inside Request Handlers (HIGH)

File: `server/domains/dtc/routes.ts` — Included in H2 fix

Changes:

- DTC integration service is now loaded via a lazy singleton pattern:

```
let _dtcIntegrationService: any = null;
async function getDtcService() {
  if (!_dtcIntegrationService) {
    const mod = await import("../dtc-integration-service.js");
    _dtcIntegrationService = mod.getDtcIntegrationService();
  }
}
```

```
    return _dtcIntegrationService;
}
```

- First call still pays the import cost; subsequent calls return cached instance

Apply the same pattern to these files (not included as full replacements since the pattern is mechanical):

- `server/domains/vessel-performance/routes/cii-routes.ts` — cache `CIIService`
 - `server/domains/compliance/routes.ts` — cache `complianceRulesEngine`
 - `server/domains/stcw-rest/routes/fatigue.ts` — cache `stcw-compliance` imports
-

H4 — Duplicate Route Registrations (HIGH)

File: `server/domains/permissions/routes.ts` — Full replacement

Changes:

- Removed second `app.get("/api/permissions/me", ...)` registration
- Route is now defined exactly once at the top of the function

Additional action needed: The duplicate insights routes in `server/domains/insights/routes.ts` should either:

1. Be removed entirely (recommended — use `insights-routes.ts` as canonical), or
 2. Delegate to the same underlying service as `insights-routes.ts`
-

H5 — RAG Streaming Auth Bypass (HIGH)

Manual patch in `server/routes/rag-routes.ts`:

Find the streaming endpoint's auth fallback (around line 370):

```
} else if (config.auth.allowHeaderOrgId) {
  // Fallback for dev mode
  orgId = (req.query.orgId as string) ||
    (req.headers['x-org-id'] as string) ||
    'default-org-id';
```

Replace with:

```
} else if (config.auth.allowHeaderOrgId && process.env.NODE_ENV === 'development'
// Dev-only fallback – blocked in production
orgId = (req.query.orgId as string) ||
    (req.headers['x-org-id'] as string) ||
    'default-org-id';
userId = (req.query.userId as string) ||
    (req.headers['x-user-id'] as string) ||
    undefined;
logger.warn('[RAG Stream] Using dev-mode auth fallback – NOT for production');
```

M1 — Widespread ‘any’ Types (MEDIUM)

No code change — this is a refactoring task.

Recommendation: In `server/routes/domain-router-registry.ts`, change:

```
getDeps: () => Record<string, any>
```

to per-domain typed interfaces. Start with the highest-traffic domains.

M2 — Mixed Error Response Shapes (MEDIUM)

Partially addressed by H1 (telemetry routes now use `withErrorHandling`).

Full fix: Create a standardized error helper in `server/lib/route-utils.ts`:

```
export function sendError(res: Response, status: number, code: string, message: s
    res.status(status).json({
        error: { code, message, ...(details && process.env.NODE_ENV === 'development'
            metadata: { timestamp: new Date().toISOString(), version: "1.0" },
        });
    }
```

Then migrate all manual `res.status(xxx).json({ error/message })` calls to use it.

M3 — Unbounded In-Memory Caches (MEDIUM)

Manual patch in `server/domains/equipment/routes.ts` and
`server/domains/integrations/routes.ts`:

Replace the hand-rolled Map caches with:

```
npm install lru-cache

import { LRUCache } from 'lru-cache';

const equipmentCache = new LRUCache<string, unknown>({
  max: 200,          // max entries
  ttl: 30 * 1000,    // 30 second TTL
});
```

Then replace `getCached/setCache/invalidateCache` with:

```
equipmentCache.get(key)          // returns undefined if expired
equipmentCache.set(key, data)
// To invalidate by prefix:
for (const k of equipmentCache.keys()) {
  if (k.startsWith(pattern)) equipmentCache.delete(k);
}
```

M4 — Compressed VPS Routes (MEDIUM)

No code change provided — this is a readability refactor.

Recommendation: Expand `server/domains/vessel-performance/routes/vps-routes.ts` to normal formatting and extract the power-STW computation into a service function.

M5 — KB Routes Catch Variable Bug (MEDIUM)

File: `server/routes/kb-routes-fix.ts` — Contains the exact patches

Changes: In `server/routes/kb-routes.ts`, find two instances of:

```
} catch (_error) {
  // ... references 'error' which is undefined
```

Change to:

```
} catch (error) {
  // ... now 'error' is properly defined
```

Locations:

1. The `router.get('/search', ...)` handler
 2. The `router.delete('/documents/:id', ...)` handler
-

L1 — Mixed Logging Patterns (LOW)

No code change — replace `console.log` with `logger.info` in:

- `server/routes/telemetry-ingestion-routes.ts` (fixed in H1)
 - `server/routes/telemetry-dlq-routes.ts` (fixed in H1)
 - Remaining: `server/routes/rag-routes.ts` , `server/routes/kb-routes.ts`
-

L2 — Inconsistent HTTP Methods (LOW)

No code change — standardization decision needed. Recommend: PATCH for status changes (acknowledge, sign, resolve), POST for creation and RPC-style actions.

L3 — Incorrect Endpoint Count Logs (LOW)

No code change — counts in `logger.info` messages are manually maintained. Consider automating or removing.

L4 — Route Registration Order Dependency (LOW)

No code change — the comment in `domain-router-registry.ts` documents the issue. Long-term fix: use more specific path prefixes for crew extensions vs scheduling.

File Manifest

File	Fixes	Action
<code>server/domains/system-admin/routes/auth-routes.ts</code>	C1	Full replacement
<code>server/routes/analytics/helpers.ts</code>	C2	Full replacement

server/domains/alerts/routes.ts	C2	Full replacement
server/routes/insights-routes.ts	C2, H4	Full replacement
server/routes/telemetry-ingestion-routes.ts	H1, L1	Full replacement
server/routes/telemetry-dlq-routes.ts	H1, L1	Full replacement
server/domains/dtc/routes.ts	H2, H3	Full replacement
server/domains/permissions/routes.ts	H4	Full replacement
server/routes/kb-routes-fix.ts	M5	Patch instructions
README.md	—	This file